

RAPPORT DE TRAVAUX PRATIQUES : ANALYSE DE TRAFIC RÉSEAU AVEC WIRESHARK

1. Introduction

Wireshark est l'outil de référence pour l'analyse de protocoles réseau. Il permet de capturer et d'examiner en détail le trafic qui transite sur une interface donnée. Dans le domaine de la cybersécurité, l'analyse réseau est une compétence fondamentale : elle permet de ne plus voir les interactions comme de simples commandes, mais de comprendre la réalité technique des échanges (paquets, flags, flux). Sans cette visibilité, il est impossible de détecter les phases de reconnaissance d'un attaquant.

2. Description des manipulations

Pour ce TP, j'ai utilisé un environnement contrôlé composé d'une machine **Kali Linux** (attaquant) et d'une machine **Ubuntu** (cible).

1. **Initialisation** : J'ai lancé Wireshark sur Kali Linux avec les privilèges `sudo` et sélectionné l'interface réseau active.
2. **Lancement de la capture** : Une fois l'écoute activée, les paquets ont commencé à défiler en temps réel.
3. **Génération du scan** : Depuis un terminal, j'ai exécuté la commande `nmap -sS 192.168.56.10`. Ce type de scan ("Stealth Scan" ou scan SYN) est conçu pour être rapide et discret.
4. **Analyse** : J'ai ensuite utilisé des filtres d'affichage pour isoler les paquets spécifiques au protocole TCP et aux tentatives de connexion.
- 5.

3. Résultats obtenus

A. Trafic brut dans Wireshark

Dès le lancement du scan, on observe un pic d'activité important dans Wireshark. Chaque ligne représente un échange entre l'attaquant et la cible.

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide						
Appliquer un filtre d'affichage ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
41	109.508023070	192.168.56.30	192.168.56.10	ICMP	98	Echo (ping) request id=0x67e7, seq=3/768, ttl=64 (reply in 42)
42	109.508350318	192.168.56.10	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e7, seq=3/768, ttl=64 (request in 41)
43	110.532673259	192.168.56.30	192.168.56.10	ICMP	98	Echo (ping) request id=0x67e7, seq=4/1024, ttl=64 (reply in 44)
44	110.532996371	192.168.56.10	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e7, seq=4/1024, ttl=64 (request in 43)
45	111.556768867	192.168.56.30	192.168.56.10	ICMP	98	Echo (ping) request id=0x67e7, seq=5/1280, ttl=64 (reply in 46)
46	111.557394695	192.168.56.10	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e7, seq=5/1280, ttl=64 (request in 45)
47	112.580429219	PCSSystemtec_0b:f7:...	PCSSystemtec_d8:d4:...	ARP	42	Who has 192.168.56.10? Tell 192.168.56.30
48	112.580679174	192.168.56.30	192.168.56.10	ICMP	98	Echo (ping) request id=0x67e7, seq=6/1536, ttl=64 (reply in 50)
49	112.580822127	PCSSystemtec_d8:d4:...	PCSSystemtec_0b:f7:...	ARP	60	192.168.56.10 is at 08:00:27:d8:d4:54
50	112.580964486	192.168.56.10	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e7, seq=6/1536, ttl=64 (request in 48)
51	112.886610604	PCSSystemtec_d8:d4:...	PCSSystemtec_0b:f7:...	ARP	60	Who has 192.168.56.30? Tell 192.168.56.10
52	112.886628362	PCSSystemtec_0b:f7:...	PCSSystemtec_d8:d4:...	ARP	42	192.168.56.30 is at 08:00:27:0b:f7:64
53	113.604336616	192.168.56.30	192.168.56.10	ICMP	98	Echo (ping) request id=0x67e7, seq=7/1792, ttl=64 (reply in 54)
54	113.604965240	192.168.56.10	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e7, seq=7/1792, ttl=64 (request in 53)
55	120.102082718	192.168.56.30	192.168.56.20	ICMP	98	Echo (ping) request id=0x67e8, seq=1/256, ttl=64 (reply in 58)
56	120.102502977	PCSSystemtec_3c:50:...	Broadcast	ARP	60	Who has 192.168.56.30? Tell 192.168.56.20
57	120.102514683	PCSSystemtec_0b:f7:...	PCSSystemtec_3c:50:...	ARP	42	192.168.56.30 is at 08:00:27:0b:f7:64
58	120.102733589	192.168.56.20	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e8, seq=1/256, ttl=128 (request in 55)
59	121.124132548	192.168.56.30	192.168.56.20	ICMP	98	Echo (ping) request id=0x67e8, seq=2/512, ttl=64 (reply in 60)
60	121.124611903	192.168.56.20	192.168.56.30	ICMP	98	Echo (ping) reply id=0x67e8, seq=2/512, ttl=128 (request in 59)
61	125.124532351	PCSSystemtec_0b:f7:...	PCSSystemtec_3c:50:...	ARP	42	Who has 192.168.56.20? Tell 192.168.56.30
62	125.124854199	PCSSystemtec_3c:50:...	PCSSystemtec_0b:f7:...	ARP	60	192.168.56.20 is at 08:00:27:3c:50:4a

B. Isolation des paquets SYN

En appliquant le filtre `tcp.flags.syn == 1`, j'ai pu isoler uniquement les tentatives d'ouverture de connexion envoyées par Nmap. On remarque une répétitivité anormale vers de nombreux ports différents.

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide						
tcp.flags.syn==1						
No.	Time	Source	Destination	Protocol	Length	Info
1930	246.152717913	192.168.56.30	192.168.56.10	TCP	58	37462 → 6669 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1931	246.152726786	192.168.56.30	192.168.56.10	TCP	58	37462 → 1068 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1932	246.152735159	192.168.56.30	192.168.56.10	TCP	58	37462 → 7200 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1933	246.152744204	192.168.56.30	192.168.56.10	TCP	58	37462 → 30718 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1934	246.152863373	192.168.56.30	192.168.56.10	TCP	58	37462 → 5985 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1935	246.152908433	192.168.56.30	192.168.56.10	TCP	58	37462 → 500 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1942	246.153027736	192.168.56.30	192.168.56.10	TCP	58	37462 → 8194 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1946	246.153191218	192.168.56.30	192.168.56.10	TCP	58	37462 → 5822 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1948	246.153247520	192.168.56.30	192.168.56.10	TCP	58	37462 → 12000 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1949	246.153264043	192.168.56.30	192.168.56.10	TCP	58	37462 → 1234 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1950	246.153273374	192.168.56.30	192.168.56.10	TCP	58	37462 → 1145 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1951	246.153281576	192.168.56.30	192.168.56.10	TCP	58	37462 → 1862 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1952	246.153290938	192.168.56.30	192.168.56.10	TCP	58	37462 → 5730 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1953	246.153376257	192.168.56.30	192.168.56.10	TCP	58	37462 → 1521 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1960	246.153495511	192.168.56.30	192.168.56.10	TCP	58	37462 → 2260 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1961	246.153514428	192.168.56.30	192.168.56.10	TCP	58	37462 → 20000 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1963	246.153608766	192.168.56.30	192.168.56.10	TCP	58	37462 → 19780 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1966	246.153742598	192.168.56.30	192.168.56.10	TCP	58	37462 → 311 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1967	246.153774310	192.168.56.30	192.168.56.10	TCP	58	37462 → 32775 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1969	246.153922437	192.168.56.30	192.168.56.10	TCP	58	37462 → 1058 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1972	246.154025656	192.168.56.30	192.168.56.10	TCP	58	37462 → 33354 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1974	246.154179151	192.168.56.30	192.168.56.10	TCP	58	37462 → 1192 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1975	246.154190488	192.168.56.30	192.168.56.10	TCP	58	37462 → 2268 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
▶ Frame 67: Packet, 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface eth0, id 0 0000 08 00 27 d8 d4 54 ▶ Ethernet II, Src: PCSSystemtec_0b:f7:64 (08:00:27:0b:f7:64), Dst: PCSSystemtec_d8:d4:54 (08:00:27:d8:d4:54) 0010 00 2c 84 1c 00 00 ▶ Internet Protocol Version 4, Src: 192.168.56.30, Dst: 192.168.56.10 0020 38 0a 92 56 00 87 ▶ Transmission Control Protocol, Src Port: 37462, Dst Port: 135, Seq: 0, Len: 0 0030 04 00 98 3e 00 00						

C. Analyse des échanges SYN / SYN-ACK / RST

Pour vérifier si un port est ouvert ou fermé, j'ai observé les drapeaux (flags) TCP :

- **Port ouvert** : On observe un échange SYN (client) -> SYN-ACK (serveur).
- **Port fermé** : On observe un échange SYN (client) -> RST (serveur).

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide									
tcp.flags.syn==1									
No.	Time	Source	Destination	Protocol	Length	Info			
2096	255.147425824	192.168.56.30	192.168.56.20	TCP	58	55290 → 21	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2097	255.147492313	192.168.56.30	192.168.56.20	TCP	58	55290 → 111	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2098	255.147512307	192.168.56.30	192.168.56.20	TCP	58	55290 → 199	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2099	255.147533513	192.168.56.30	192.168.56.20	TCP	58	55290 → 22	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2100	255.147636953	192.168.56.20	192.168.56.30	TCP	60	135 → 55290	[SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460		
2107	255.147912405	192.168.56.30	192.168.56.20	TCP	58	55290 → 113	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2108	255.147966013	192.168.56.30	192.168.56.20	TCP	58	55290 → 554	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2111	255.150404047	192.168.56.30	192.168.56.20	TCP	58	55290 → 5900	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2112	255.150433652	192.168.56.30	192.168.56.20	TCP	58	55290 → 23	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2113	255.150461894	192.168.56.30	192.168.56.20	TCP	58	55290 → 3389	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2114	255.150471176	192.168.56.30	192.168.56.20	TCP	58	55290 → 80	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2115	255.150481781	192.168.56.30	192.168.56.20	TCP	58	55290 → 139	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2116	255.150490293	192.168.56.30	192.168.56.20	TCP	58	55290 → 1723	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2117	255.150583270	192.168.56.30	192.168.56.20	TCP	58	55290 → 2288	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2118	255.150613986	192.168.56.30	192.168.56.20	TCP	58	55290 → 9002	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2119	255.150644280	192.168.56.30	192.168.56.20	TCP	58	55290 → 26214	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2120	255.150657793	192.168.56.30	192.168.56.20	TCP	58	55290 → 10566	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2124	255.150822474	192.168.56.30	192.168.56.20	TCP	58	55290 → 464	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2126	255.150894500	192.168.56.20	192.168.56.30	TCP	60	139 → 55290	[SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460		
2134	255.151145311	192.168.56.30	192.168.56.20	TCP	58	55290 → 5280	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2135	255.151182642	192.168.56.30	192.168.56.20	TCP	58	55290 → 2968	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
2136	255.151267598	192.168.56.30	192.168.56.20	TCP	58	55290 → 3995	[SYN] Seq=0 Win=1024 Len=0 MSS=1460		
Frame 2114: Packet, 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface eth0, id 0 0000 08 00 27 3c 50 4a 08 00 27 Ethernet II, Src: PCSystemtec_0b:f7:64 (08:00:27:0b:f7:64), Dst: PCSystemtec_3c:50:4a (08:00:27:3c:50: 0010 00 2c d6 ab 00 00 3b 06 b7 Internet Protocol Version 4, Src: 192.168.56.30, Dst: 192.168.56.20 0020 38 14 d7 fa 00 50 07 71 7c Transmission Control Protocol, Src Port: 55290, Dst Port: 80, Seq: 0, Len: 0 0030 04 00 46 68 00 00 02 04 95									

4. Analyse technique

- **Qu'est-ce qu'un paquet SYN ?**

Le paquet SYN (Synchronize) est le premier paquet envoyé lors de l'établissement d'une connexion TCP (le "Three-way handshake"). Il sert à demander l'ouverture d'une session.

- **Comment détecter un port ouvert ?**

Un port est considéré comme ouvert si, après l'envoi d'un SYN, la cible répond par un **SYN-ACK** (Synchronize-Acknowledgment). Dans le cadre d'un scan -ss, l'attaquant envoie alors un RST pour fermer la connexion sans la finaliser.

- **Comment reconnaître un scan réseau ?**

Un scan se reconnaît par trois facteurs clés :

1. Un volume élevé de paquets SYN en un temps très court.
2. Une provenance unique (même IP source).
3. Une destination vers une multitude de ports différents (balayage).

5. Utilisation avancée de Wireshark

Filtre Avancé	Utilité	Exemple concret
<code>ip.addr == [IP]</code>	Isole tout le trafic entrant ou sortant d'une machine précise.	<code>ip.addr == 192.168.56.10</code>
<code>tcp.port == [Port]</code>	Filtre le trafic sur un service spécifique (ex: HTTP, SSH).	<code>tcp.port == 80</code>
<code>http.request.method == "POST"</code>	Détecte les envois de données (souvent utile pour les formulaires).	Identifier une tentative d'exfiltration ou de brute-force.

6. Lien avec la cybersécurité en entreprise

Dans un **SOC (Security Operations Center)**, les analystes utilisent Wireshark pour effectuer du "Deep Packet Inspection". Lorsqu'une alerte IDS/IPS signale un comportement suspect, Wireshark permet de confirmer la nature de l'attaque en inspectant le contenu réel des paquets. Cela permet de différencier une erreur de configuration d'une véritable intrusion, comme un scan de vulnérabilités ou une tentative d'exploitation.

7. Conclusion

Ce TP m'a permis de comprendre que derrière une simple commande Nmap se cache une mécanique précise de drapeaux TCP.

- **Appris :** Interprétation des flags TCP et utilisation des filtres Wireshark.
- **Difficultés :** Identifier le bon paquet de réponse parmi le flux massif de données initial.
- **Limites de Wireshark :** C'est un outil passif d'analyse ; il ne bloque pas les attaques et peut être difficile à utiliser sur des réseaux à très haut débit sans filtres préalables.

Désormais, je ne vois plus un scan comme une simple commande, mais comme une suite logique d'interactions réseau qu'un analyste SOC doit savoir identifier pour protéger l'infrastructure.